PROJECT SHEET 2 · EVERY FINDING REPRODUCED IN SIMULATION

Bugs in the Repo — Evidence & Fixes

Ran `make all_tests SIM=iverilog` on the repo as-is, then directed probes. Fixes below were re-run: 6/6 pass for real, and failures now fail.

DUT design bug
TB testbench bug
TOOL simulator limitation

#1 Regression says PASS while the scoreboard logs FAILs

TB · most important

Evidence (repo as-is): `ALL TESTS DONE: PASS=6 FAIL=0` — yet the logs show scoreboard failures in 5 of 6 tests: read 7/1, rw 8/8, stress 30/2, error 1/1, wr_rd_chk 2/1 (pass/fail).

Root cause: `scoreboard_report()` only `$display`s "TEST FAILED"; `vvp` still exits 0, and the Makefile judges a test by exit code. So **this testbench cannot fail**.

Interview gold: "How do you know your test passed?" Answer: a non-zero exit / UVM_ERROR count checked by the regression script, and a check that the scoreboard compared **more than zero** items.

FIX (VERIFIED: FAILING RUN NOW EXITS 1 → MAKE REPORTS FAILED)

```
if (sb_fail == 0 && sb_pass > 0)
  $display("[SB] *** TEST PASSED ***");
else begin
  $display("[SB] *** TEST FAILED ***");
  $fatal(1, "scoreboard failures");   // non-zero exit
end
// UVM: fail in report_phase if UVM_ERROR count > 0,
// and grep the log for "UVM_ERROR :    0" in regression
```

#2 Deadlock when write data arrives before the address

DUT · protocol violation

In `WR_ADDR` the slave holds **WREADY high** but only looks at AWVALID. A master that sends W first completes the W handshake — the slave **drops the data**. When AW arrives the FSM goes to `WR_DATA` and waits forever for a W beat that already happened.

Evidence: directed probe (W first, then AW, BREADY high) → `NO BVALID after 50 cycles`.

Why the TB missed it: the driver always raises AWVALID and WVALID in the **same cycle**. W-before-AW is legal AXI and the stimulus never produced it — a **stimulus coverage hole**, not a checker problem.

FIX: ACCEPT AW AND W INDEPENDENTLY (VERIFIED: BVALID ARRIVES)

```
assign awready = !aw_got && !bvalid;   // from own state only
assign wready  = !w_got  && !bvalid;
always_ff @(posedge aclk or negedge aresetn)
  if (!aresetn) {aw_got, w_got, bvalid} <= '0;
  else begin
    if (awvalid && awready) begin aw_addr <= awaddr; aw_got <= 1; end
    if (wvalid  && wready)  begin w_data <= wdata; w_strb <= wstrb; w_got <= 1; end
    if (aw_got && w_got && !bvalid) begin
      if (in_range) mem[idx] <= apply_strobe(mem[idx], w_data, w_strb);
      bresp  <= in_range ? 2'b00 : 2'b10;     // also fixes #3
      bvalid <= 1;  aw_got <= 0;  w_got <= 0;
    end
    if (bvalid && bready) bvalid <= 0;
  end
```

#3 No SLVERR — out-of-range addresses alias

DUT

The header comment and the UVM scoreboard both expect **SLVERR** above 256 words, but the RTL indexes with `(addr >> 2) % 256` and always returns OKAY.

Evidence: write DEADBEEF to `0x400` → `BRESP = 00`; then read `0x000` → `DEADBEEF`. The out-of-range write **corrupted word 0**.

error_test "passed" because it wrote and read the same aliased address — a **self-consistent but wrong** result. Classic reason to check against a spec-based model, not a DUT echo.

Fix: `in_range = addr[31:2] < MEM_DEPTH`. Write: skip the memory update, BRESP = 10. Read: RDATA = 0, RRESP = 10. Scoreboard expects SLVERR and doesn't update the model. Verified: both return `10`, word 0 untouched.

#4 Monitor records a bogus X transaction at start-up

TB

Monitor loops on `while (!(bus.arvalid && bus.arready))`. At start ARVALID is **X**, ARREADY is 1 → condition is X → a `while` treats X as false and **exits immediately**, capturing a read with `addr = X`.

Evidence: `[SB] FAIL READ addr=0xxxxxxxxx` as the first scoreboard entry in 5 tests.

Fix (verified): 4-state compare `while (!(bus.arvalid === 1'b1 && bus.arready === 1'b1))`, and drive all master VALIDs to 0 from time 0, not after reset.

Concept: X-pessimism / X-optimism. `if` and `while` treat X as false; `==` returns X; `===` returns 0/1. Add an SVA "no X on VALID after reset".

#5 Scoreboard model corrupts data on iverilog

TOOL

`sb_mem[idx][b*8+:8] = tr.data[b*8+:8]` — a variable part-select **on a class property**. iverilog returns the low byte for every lane.

Evidence: 5-line repro: `c.data = 40895E81` → model got `81818181`. Same pattern in rw_test: `exp=0x81818181 got=0x40895e81` — the DUT was right, the **checker** was wrong.

Fix (verified): copy into locals first: `d = tr.data; st = tr.strb;` then part-select `d`.

Lesson: prove the checker can catch a **known-bad DUT**, and when a mismatch appears, suspect the model as much as the RTL.

#6 The UVM files have never been compiled

TB

Problem	Effect
Makefile compiles only pkg + interface + tb_top	tb/axi4_agent, driver, env... are dead code
No `import uvm_pkg::*`, `run_test()` or `config_db::set`	Nothing would start; driver's `get` would hit uvm_fatal
Interface has no `driver_cb` / `monitor_cb`, no modports, no SVA (header claims all three)	Driver and monitor don't compile
`uvm_analysis_imp_write` used without `` `uvm_analysis_imp_decl(_write) ``	Coverage class doesn't compile
Typo `bins55_pat` (missing space)	Syntax error in cg_wr_data
Two different `axi4_seq_item` classes	Name clash if both compiled
error_seq inline range conflicts with `c_addr_legal`	randomize() fails → uvm_fatal

#7 Coverage & stimulus gaps

TB

- **WSTRB:** comment says "all 15 patterns", covergroup has 10 bins, constraint can only generate 7 → 0010, 0100, 1000, 0111, 1110 are **unreachable**; coverage can never close.
- **No protocol coverage:** handshake order (AW first / W first / same cycle) and back-pressure length are never measured — exactly where bug #2 hid.
- **READY never randomised:** driver raises BREADY / RREADY at once; slave hold behaviour is barely exercised.
- **SLVERR bins** unreachable while #3 exists — a coverage hole that would have flagged the bug if anyone had looked.
- **UVM scoreboard skips** reads of never-written words — but the RAM resets to 0, so those could be checked.
- **End of test** by fixed `#100ns` instead of waiting for the scoreboard to drain.

Coverage only tells you what you **asked** about. A 100% number on a weak plan proves little.

★ Turn this into your STAR story

SITUATION

AXI4-Lite slave RAM, self-checking TB, regression green 6/6.

TASK

Make sure green actually meant correct before calling it done.

ACTION

- Read the logs, not the summary → found FAIL lines hidden behind exit code 0.
- Root-caused each: X-pessimism in the monitor, a simulator part-select bug in the model.
- Added W-before-AW stimulus → **DUT deadlock**; probed out-of-range → **aliasing, no SLVERR**.
- Fixed RTL, made failures fatal, added SVA + handshake-order coverage.

RESULT

2 real DUT bugs fixed, a regression that can fail, and coverage that measures the protocol, not just addresses.

Do the fixes in the repo first — then every line of this story is true and you can show the commits.